

You are **Claude Opus**, acting as an **expert full-stack simulation engineer**. Build a **4-file** “no frameworks” web + FastAPI projectile simulation **from scratch**, designed to be **near-identical** to the described `projectile.js` behaviors and contracts.

You must output **TWO PHASES** in a single response:

- 1) **PHASE 1 — PLAN (NO CODE)**
- 2) **PHASE 2 — CODEGEN (ACTUAL CODE for all 4 files)**

You must follow all constraints exactly. Do **not** ask questions. Do **not** add extra files. Do **not** add frameworks. Do **not** omit required DOM IDs, endpoints, keys, or behaviors.

FILES (exact filenames)

You will generate these **exact** files in PHASE 2:

- 1) `index.html`
- 2) `styles_projectile.css`
- 3) `projectile.js`
- 4) `main_projectile.py`

CRITICAL CONSTRAINTS (NON-NEGOTIABLE)

A) Server injection contract (MUST)

- `server` injects this **in** `<head>`:

- `window.__SIM_CFG__ = {...}`

- `projectile.js` **MUST** read `window.__SIM_CFG__` **BEFORE** creating any Matter bodies (mass + initial conditions depend on it).

- Accepted injected keys (all optional):

- `dropHeight_m`, `launchSpeed_mps`, `launchAngle_deg`, `ballMass`, `airEnabled`, `forceEnabled`, `velocityEnabled`, `motionEnabled`

- Enforce exclusivity:
 - Exactly **ONE** of `(forceEnabled, velocityEnabled, motionEnabled)` must be true.
 - If ambiguous or none/invalid: default to:
 - `forceEnabled=true`, `velocityEnabled=false`, `motionEnabled=false`
- `accelerationEnabled` is controlled **only** by UI checkbox (`#accelerationCheckbox`) and is exclusive with the other three UI modes (see Mode rules below). It is **not** injected.

B) Backend contract (MUST)

- FastAPI base: `http://127.0.0.1:8000`

Endpoints (exact)

- 1) `GET /projectile?t=...&mass=...&height=...&speed=...&angle=...&eps=...&g=...&vectors=...`
 - `vectors` may be ignored but must not break.
- 2) `GET /projectile\_series?mass=...&height=...&speed=...&angle=...&dt=...&eps=...&g=...`

Physics rules (backend)

- Backend is **NO AIR RESISTANCE** physics. Always ideal projectile / free-fall after release.
- The **air** checkbox is visual only in frontend (draw drag vectors) and must **NOT** change backend states.

Backend returned keys (MUST MATCH EXACTLY)

Every state object returned/used by frontend must include these keys exactly:

- `time\_s, x, y, vx, vy, ax, ay, Fg, Fn, v, onGround, phase, y\_raw`

/projectile_series payload shape (MUST)

Response JSON must include:

- `meta` object with exactly:
 - `vx0, vy0, t\_land, t\_apex, y\_apex, x\_land, t\_pre, dt`
- `series`: array of **state objects** (each with the required keys)
- `impact`: impact snapshot (impact moment)

- `rest`: rest snapshot (ball at rest on ground)
- `events` object with keys:
 - `start, apex, impact, rest`

Special backend behaviors (MUST)

- Rounding rule:
 - `round\_floats()` rounds **TIME_KEYS** to **6 decimals** and all other floats to **2 decimals**.
 - TIME_KEYS should include at least: `time\_s` (and any meta times: `t\_land, t\_apex, t\_pre`).
- Robust landing logic:
 - Compute landing, apex, and rest cleanly.
- “On-ground-at-start” handling:
 - If `height=0` and `vy0≈0` at `t=0`, return a state showing **$F_n = F_g$ immediately** (rest).

C) HTML ↔ JS contract (MUST)

Your `index.html` MUST include these elements / IDs exactly because `projectile.js` depends on them:

Simulation mount

- `div#scene` (Matter.Render injects canvas inside here)

Numeric inputs

- `input#massInput` (number)
- `input#heightInput` (number)
- `input#velocityInput` (number)
- `input#angleInput` (number)

Mode checkboxes (exclusive)

- `input#forceCheckbox`
- `input#vectorCheckbox`
- `input#motionCheckbox`
- `input#accelerationCheckbox` ← MUST EXIST (required by accel mode behaviors)

Air resistance checkbox (visual only; only meaningful in FORCE mode; may be hidden otherwise)

- `input#airCheckbox`

Buttons

- `button#pauseBtn`
- `button#restartBtn`
- `button#frameStartBtn`
- `button#frameApexBtn`
- `button#frameEndBtn`

Scrubber

- `input#timeScrubber` (type=range)

Speed buttons

- `container#speedButtons` containing multiple `button.speed-btn` elements, each having `data-speed` values:
 - `0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0`

Graph canvases (Chart.js)

- `canvas#graph-x-t`
- `canvas#graph-y-t`
- `canvas#graph-vx-t`
- `canvas#graph-vy-t`
- `canvas#graph-Fg-t`
- `canvas#graph-Fn-t`
- `canvas#graph-Fg-m`
- `canvas#graph-ax-t`
- `canvas#graph-ay-t`

Key / legend DOM

- `div#keyBox`
- `div#keyForces`, `div#keyVelocity`, `div#keyMotion` (sections)
- `tr#keyRowFd` (air row toggled)
- `tr#keyRowTrail` (trail row; can exist always)

Required script includes

- Include **Chart.js** CDN in ``<head>``
- Include **Matter.js** CDN before `projectile.js` at end of body
- Load `projectile.js` last

D) projectile.js “NEAR-IDENTICAL” behavioral requirements (MUST)

1) Structure / wrapping (MUST)

- Wrap everything in:
 - ``window.addEventListener("DOMContentLoaded", () => { ... })``
- Matter destructure (exact names):
 - ``Engine, Render, Runner, World, Bodies, Body, Events``

2) Constants before lets (MUST)

- All constants declared before any ``let``.
- Required constants (exact names + intent):
 - ``canvasWidth=800`, `canvasHeight=700`, `g=9.81``
 - ``groundHeight_px=40`, `ballRadius=10``
 - ``maxRange_m=1200`, `maxHeight_m=650``
 - ``FETCH_INTERVAL=200``
 - ``TRAIL_MAX_POINTS=600`, `TRAIL_MIN_DIST_PX=3``
 - ``GROUND_CENTER_Y`` (computed), ``GROUND_TOP_Y`` (computed)
 - ``CHART_UPDATE_EVERY=2``
 - ``PERF`` object with:
 - ``dotOnlyWhilePlaying, dotHz, maxLinePoints``
 - ``LINE_KEYS`` array:
 - ``["x_t", "y_t", "vx_t", "vy_t", "Fg_t", "Fn_t", "ax_t", "ay_t"]``

3) Functions placement & side effects rule (CRITICAL)

- Define **all** helper functions (math, fetch, charts, overlays, UI, scrubber) **above** ``initRun()``.
- **Only** ``initRun()`` may perform side effects:
 - create engine/render/runner
 - create charts
 - bind DOM listeners
 - start timers/loops
- Config normalization from ``window.__SIM_CFG__`` must occur **before** any Matter/Chart creation.
- You may declare ``let engine, render, runner, ...`` early, but do not create them until ``initRun()``.

4) Modes (MUST)

- forceEnabled:
 - Draw **Fg** arrow always
 - Draw **Fn** only when “ground should show”
- velocityEnabled:
 - Draw **Vx**, **Vy**, and resultant **V** arrows
- motionEnabled:
 - Draw **dotted trail** + **dashed x/y coordinate arrows**
- airEnabled:
 - Draw drag decomposition triangle (**Fd**, **Fdx**, **Fdy**) **ONLY** in forceEnabled mode
 - Visual only (no backend changes)
- accelerationEnabled:
 - Draw **ay** arrow down
 - Controlled by UI checkbox; exclusive UI mode like the others.

5) Graph visibility logic (MUST)

- Use ``GRAPH_CANVAS`` map: chart keys → canvas IDs.
- ``updateGraphVisibility()`` MUST set ``visibleKeys = showSet``.

Visibility sets:

- Force mode shows: ``Fg_t, Fn_t, Fg_m``
- Velocity mode shows: ``vx_t, vy_t``
- Motion mode shows: ``x_t, y_t, vx_t, vy_t, ax_t, ay_t``
- Acceleration mode shows: ``ax_t, ay_t``

6) Chart.js dataset structure (CRITICAL)

Each line chart has **2 datasets**:

- ``dataset[0]``: history line
 - ``showLine: true``
 - ``pointRadius: 0`` except initial seed point if desired
- ``dataset[1]``: moving dot (“Start Dot”)
 - hidden by default
 - ``showLine: false``

Rules:

- ``setMovingDotAtState(st)`` updates ``dataset[1]`` for visible charts **without rebuilding history**.
- Fn dot logic must **snap to landing** at end:

- at `tLand`, `Fn = Fg`.

7) Backend pre-impact handling at end/scrub edge (CRITICAL)

- Must implement:
 - `preLandingTime() = tLand - preImpactEps()`
- Visuals/vectors at end/right edge must use `preLandingTime` so `vx/vy` don't jump to 0.
- Graph/history at end includes true landing point at `tLand` where `Fn=Fg`.

8) Scrubber behaviors (CRITICAL)

- Slider:
 - `step = 0.01`
 - bounds sync to landing time, max snapped to `ceil(tLand/0.01)\*0.01`
- Scrubbing:
 - begins: internal pause ****without changing Pause button label**** until release
 - ends:
 - left edge: behaves like Restart-at-start (clear charts, seed $t=0$)
 - right edge: knob snaps to true end; visuals use `preLandingTime()`, graphs use `tLand`
- Must support pointer + mouse + touch events:
 - with `beginScrub()` / `endScrub()`

9) Freeze frame buttons (MUST)

- Start:
 - dot-only snapshot at `t=0` (no line history)
- Apex:
 - rebuild history up to `tApex`, pause at apex
- End:
 - visuals at `preLandingTime()`
 - graphs rebuilt to `tLand`
 - mark landed and show Fn

10) Restart behavior (MUST)

- Hard reset pause label to `"Pause"`
- Correctly lock/unlock buttons/states
- Re-fetch series once
- Reset charts fast, then seed `t=0` into dataset[0]

11) Fetching (MUST)

- ``fetchTrajectorySeries()`` calls ``/projectile_series`` with:
 - ``dt=0.02``
 - ``eps=preImpactEps()``
- ``ensureBoundarySamples()`` guarantees:
 - series includes ``t=0`` exactly
 - series includes an impact/landing point at ``tLand`` where ``Fn=Fg``
 - keep pre-impact velocities at that landing point so velocity graphs do NOT drop to 0 at `tLand`
 - normalize ``trajCache.rest`` at ``tLand``:
 - ``Fn=Fg``, ``vx=vy=v=0``, ``ax=ay=0``

12) Matter engine (MUST)

- ``engine.gravity.y = 0`` (backend controls motion)
- Bodies:
 - ball circle
 - ground rectangle
 - added via ``World.add``
- Render:
 - ``Render.create({ element: #scene, ... })``
 - include background grid (drawn in `afterRender` or `background style`)
- Events:
 - ``Events.on(engine,"beforeUpdate", ...)`` drives time ``t_s`` and uses ``Body.setPosition(...)`` from backend series
 - ``Events.on(render,"afterRender", ...)`` draws arrows/trail/labels

E) Python implementation requirement (MUST)

``main_projectile.py`` must be functionally identical to the backend described above, including:

- CORS allow all origins
- ``round_floats()``:
 - 2 decimals for all floats except `TIME_KEYS` at 6 decimals
- landing logic: rest state, impact snapshot, apex metadata
- on-ground-at-start handling: `height=0` and `vy0≈0` $\Rightarrow$ `Fn=Fg` immediately

PHASE 1 OUTPUT FORMAT (PLAN — NO CODE)

Output exactly these sections, in order:

- 1) **Architecture overview** (what each file does)
- 2) **Hard requirements checklist** (must be satisfied)
- 3) **HTML contract** (every required DOM ID/element and how JS uses it)
- 4) **projectile.js structure plan**
 - Ordered sections (in order), constants before lets
 - Must include beforeUpdate + afterRender
 - Must include all named functions/behaviors
 - Major functions listed + what DOM IDs they touch
 - Key state variables listed
- 5) **FastAPI plan** (endpoints, rounding rules, payload structure)
- 6) **Test plan** (how to verify behavior)

Also include a short list titled:

- **Do not break these invariants (JS)**

The plan must be explicit enough that a second engineer could implement from it.

PHASE 2 OUTPUT FORMAT (CODEGEN — ACTUAL CODE)

After PHASE 1, generate the actual code for all 4 files.

Output rules (MANDATORY)

- Output **4 separate code blocks**, each preceded by:
 - `### <filename>`
- Use triple backticks with language tags:
 - ``html`, `css`, `javascript`, `python``
- Provide **ONLY** these four files + at the very end a short **RUN INSTRUCTIONS** section.
- Do not omit anything needed to run locally.
- Hardcode backend base URL as:
 - ``http://127.0.0.1:8000`` exactly.

projectile.js ordering constraint (MANDATORY)

Keep this ordering strictly:

- 1) DOMContentLoaded wrapper
- 2) Matter destructure
- 3) **constants first**
- 4) ``lets`` (state vars)
- 5) ``applyInjectedSimCfg()`` / ``normalizeCfg()`` (IIFE or immediate call) — must happen before any Matter/Chart creation
- 6) helper functions (math/fetch/charts/overlay/ui/scrubber) — all function declarations above `initRun`
- 7) chart creation helpers (but chart creation itself only in `initRun`)
- 8) render overlay functions
- 9) `beforeUpdate` handler function
- 10) `afterRender` handler function
- 11) controls + mode toggles + speed buttons logic
- 12) scrubber (`beginScrub/endScrub` + `pointer/mouse/touch`)
- 13) ``initRun()`` at the bottom, the ONLY place that:
 - creates engine/render/runner
 - creates bodies
 - creates charts
 - binds event listeners
 - starts timers/loops

index.html requirements (MUST include these IDs)

Must include at minimum (exact IDs):

- ``div#scene``
- inputs: ``#massInput, #heightInput, #velocityInput, #angleInput``
- checkboxes: ``#forceCheckbox, #vectorCheckbox, #motionCheckbox, #accelerationCheckbox, #airCheckbox``
- buttons: ``#pauseBtn, #restartBtn, #frameStartBtn, #frameApexBtn, #frameEndBtn``
- ``input#timeScrubber`` type=range
- ``div#speedButtons`` with ``button.speed-btn`` and ``data-speed`` values:
 - 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0
- canvases:
 - ``graph-x-t, graph-y-t, graph-vx-t, graph-vy-t, graph-Fg-t, graph-Fn-t, graph-Fg-m, graph-ax-t, graph-ay-t``
- key box:
 - ``div#keyBox``, sections ``#keyForces #keyVelocity #keyMotion``

- rows with ids `#keyRowFd` and `#keyRowTrail`
- Load Chart.js CDN in ``
- Load Matter.js CDN before projectile.js at end of body
- Load projectile.js last

CSS requirements

- Must not break layout
- Must ensure `#scene` has a fixed visible height and Matter canvas fills it
- Must style graph wrappers (if used) so canvases are visible

projectile.js functional requirements recap (MUST)

- Wrap in DOMContentLoaded
- Matter destructure and Engine.create with gravity.y=0
- Apply `window.\_\_SIM\_CFG\_\_` BEFORE creating bodies; enforce mode exclusivity
- Use exact UI IDs via `document.getElementById`
- Charts: `createLineChart()` creates Chart with TWO datasets (history + moving dot)
- Graph visibility uses `GRAPH\_CANVAS` map and `updateGraphVisibility()` sets `visibleKeys`
- Backend calls:
 - `/projectile` for rare point fetch
 - `/projectile\_series` for trajCache
- EnsureBoundarySamples:
 - guarantee $t=0$ exists exactly
 - guarantee landing point at t_{Land} where $F_n=F_g$
 - keep pre-impact velocities at landing point (so V graphs do not drop at t_{Land})
 - rest snapshot has $v_x=v_y=v=0$ and $a_x=a_y=0$
- Scrubber step=0.01, max snapped to $\text{ceil}(t_{Land}/0.01)*0.01$
- beginScrub pauses internally without toggling pause label; endScrub handles left/right edge rules
- Freeze buttons:
 - Start dot-only at $t=0$
 - Apex rebuild to t_{Apex} then pause
 - End visuals at preLandingTime , graphs to t_{Land} , landed state with F_n shown
- Render overlays in afterRender:
 - labeled Start/Apex/End dots with special-case labeling for $0^\circ/90^\circ$ and $\text{height}>0$
 - Force arrows: F_g always; F_n only on ground/end/right edge
 - Air drag triangle (visual only) when airEnabled AND force mode
 - Velocity vectors in velocity mode

- Trail + coordinate arrows in motion mode
- Acceleration arrow in acceleration mode
- Restart:
 - resets pause button text to "Pause"
 - clears dot datasets and line datasets correctly
 - refetches series once and seeds $t=0$
- Speed buttons:
 - setPlaybackSpeed updates engine.timing.timeScale
 - toggles `.active`

FastAPI requirements recap (MUST)

- Implement:
 - CORS allow_origins=["*"]
 - TIME_KEYS rounding to 6 decimals; other floats to 2 decimals
 - `\_meta()`, `\_flight\_state()`, `\_rest\_state()`, `\_impact\_snapshot()`
- `/projectile` endpoint:
 - if start on ground and $v_{y0} \approx 0$ at $t=0 \Rightarrow$ return rest with $F_n = F_g$
 - if $t \geq t_{\text{land}} \Rightarrow$ return impact snapshot ($F_n = F_g$, $y=0$, velocities from pre-impact)
 - else return flight state
- `/projectile\_series` endpoint:
 - returns payload with meta, series, impact, rest, events
 - include apex sample when available
 - includes $t_{\text{pre}} = t_{\text{land}} - \text{eps}$
 - dt default 0.02, eps default 0.0
- No air resistance in backend physics

FINAL OUTPUT REQUIREMENT

After the 4 code blocks, output a short section:

****RUN INSTRUCTIONS****

- how to run FastAPI:
 - `uvicorn main\_projectile:app --reload --port 8000`
- how to open/serve `index.html`
- what should appear visually

Now proceed:

- 1) PHASE 1 — PLAN (no code)
- 2) PHASE 2 — CODEGEN (4 files)

Do not add commentary outside these required outputs.